

Assignment 1 – Report

Name: Hasan Amjad

Roll No.: 25280042

GitHub repository: <https://github.com/HasanAmjad/autoformer-exogenous-forecasting>

Summary. This report presents the work done for both tasks of AI651 Assignment 1. In Task 1, four forecasters are compared on a synthetic vibration dataset: shared, sensor-specific and period-routed ridge, and Raw Attention. The report covers their errors on established and held-out stations, why the forecasting rule must depend on context, and how decomposition separates slow drift from repetition. In Task 2, an Autoformer is built to forecast 168 steps of an anonymised hourly series. Its periodic structure is recovered, a chronological validation protocol is set up, and an ablation shows that the covariates help only when they cover the forecast horizon.

Task 1 – Forecasting Across Heterogeneous Sensors

All numbers below come from a single full run (preset full; data seed 0; model seed 0; Apple GPU; 650/105/105 train/validation/test origins). Unless stated otherwise, errors are validation RMSE in m/s^2 ; “established” means Stations 1–3 and “held-out” means Station 4.

Response 1A – Predicted vs. measured ordering

Table 1: Predicted ordering, written before running Output 1.1.

Population	Prediction (best $\rightarrow$ worst)
Established stations	Raw Attention > Period-routed > Shared > Sensor-specific
Station 4 forecast*	Raw Attention > Period-routed > Shared ridge

*Sensor-specific ridge isn’t part of the Station 4 forecast, since it requires a matrix fitted on that station’s own training data, which isn’t available in our case as Station 4 is held out.

The initial intuition was that Raw Attention may lead, since it is basically a neural network that builds its own mixing rule from each window. It can handle changing data because its attention weights are recomputed from the current window, so the same learned projections produce a different mixing rule for each operating speed.

Table 2: Output 1.1 – validation RMSE (m/s^2) by population and product.

Model	Established (Stations 1–3)				Held-out (Station 4)			
	All	A	B	C	All	A	B	C
Shared ridge	0.767	0.805	0.854	0.621	0.876	0.909	0.982	0.713
Sensor-specific ridge	0.768	0.804	0.852	0.630	n/a	n/a	n/a	n/a
Period-routed ridge	0.166	0.181	0.164	0.152	0.173	0.184	0.182	0.150
Raw Attention	0.436	0.412	0.462	0.433	0.659	0.677	0.619	0.678

When the models were actually run, Output 1.1 told a different story: Period-routed ridge

and Raw Attention swap places in both the established and Station 4 forecasts. On established stations, Period-routed ridge reaches an RMSE of 0.166 against 0.436 for Raw Attention; on Station 4 it is 0.173 against 0.659. The same ordering holds for every product in both populations, with Period-routed ridge best on Products A, B and C alike (0.150–0.184).

As anticipated, errors get worse when moving to the held-out Station 4, and the trend is similar in both cases, though it gets much worse for Raw Attention ($0.436 \rightarrow 0.659$, +51%) compared with Period-routed ridge ($0.166 \rightarrow 0.173$, +4%) and Shared ridge ($0.767 \rightarrow 0.876$, +14%).

Regarding why the RMSE for Period-routed ridge is the lowest, our understanding is that it uses the one thing that changes between windows, the period. It first estimates how fast the machine is running, then picks a model fitted only on windows with a similar speed.

Raw Attention didn't do as well as I initially thought because it has to learn this speed dependence from the data on its own. Meanwhile, the slow drift dominates the raw input, and every pair of positions is scored separately instead of being grouped by their spacing. Its large drop on Station 4 also suggests it partly learned station-specific details rather than the speed structure.

Shared and Sensor-specific ridge have practically the same RMSE on established stations (0.767 vs. 0.768). The reason is that the underlying forecasting rule depends on the machine's speed, not on which station produced the data. Splitting by station just leaves each formula with less data, and each one still has to average across speeds.

Revised ordering (both populations). Period-routed ridge > Raw Attention > Shared ridge $\approx$ Sensor-specific ridge (the last is unavailable for Station 4).

Response 1B – Why the forecasting rule must depend on context

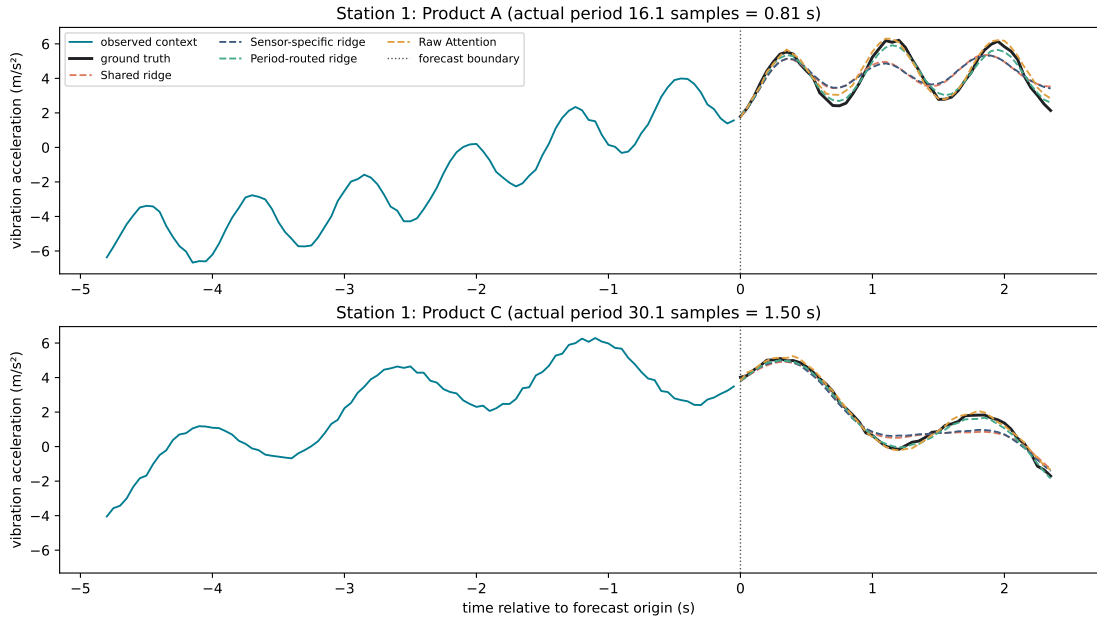


Figure 1: Output 1.2 – Station 1 under Product A ($P \approx 16.1$) and Product C ($P \approx 30.1$).

The single matrix $\widehat{W}$ in Eq. (8) is one fixed formula, fitted once on all training windows. These windows contain waves of very different speeds, with periods anywhere from 14 to 36 samples.

A fast wave (16 samples) and a slow wave (30 samples) need to be continued differently over the next 48 steps. So, one formula can't fit both and ends up giving an average of the two.

This averaging is visible in Figure 1. For Product A, the shared and sensor-specific ridge forecasts have flattened peaks and start falling behind the true wave in the second half of the horizon (window RMSE 0.779 and 0.787).

Period-routed ridge uses a formula fitted only on waves of a similar speed, so it follows the true wave closely (0.232). When two waves that are out of step are averaged, they partly cancel each other out. That is why the shared forecast becomes weaker (smaller peaks), not just shifted in time.

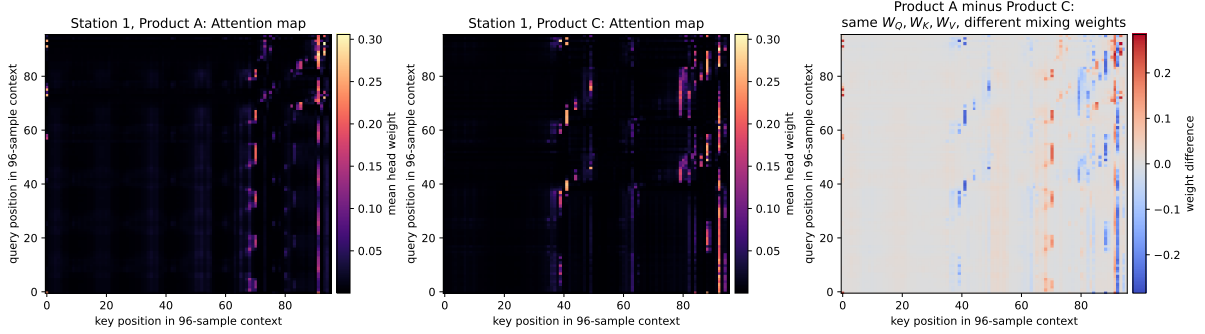


Figure 2: Output 1.3 – Attention maps for two Station 1 contexts and their difference.

Attention uses the same learned weights (W_Q, W_K, W_V) for every window. However, it computes its attention scores ($QK^\top / \sqrt{d_h}$) from the current window, so each new input produces a new set of weights.

Figure 2 shows this: the Product A and Product C maps focus on different positions (mean absolute weight difference 0.013, maximum 0.284). So, attention mixes each window in its own way, whereas ridge applies the same $\widehat{W}$ to every window. This only shows that attention adapts to the input. It doesn't prove that attention has found the actual period of the wave.

Response 2 – Separating slow structure from repetition

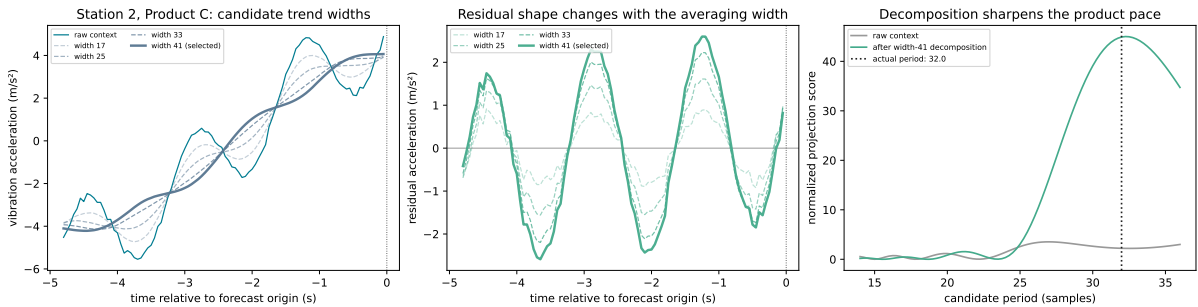


Figure 3: Output 2.1 – trend and remainder for candidate widths (Station 2, Product C).

Choosing the width m . When the averaging width is small, the trend follows part of the vibration. At $m = 17$ the trend wiggles, and the remainder is weak (std 0.578). As m grows, the trend becomes a smooth ramp and the remainder becomes a clean, full-size wave (std 1.572 at $m = 41$). After decomposition, the period score peaks sharply at the true period (32), while the raw signal gives almost no peak.

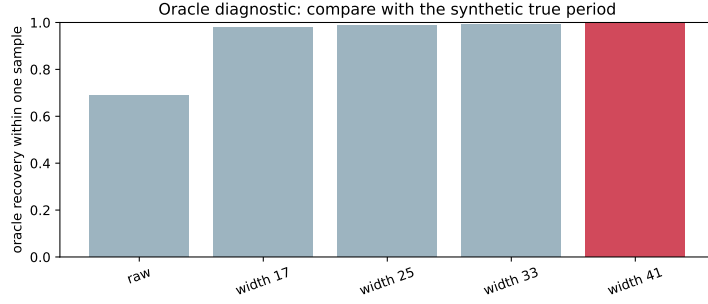


Figure 4: Output 2.2 – oracle period recovery (training data only).

Table 3: Outputs 2.1–2.2 – effect of the averaging width m .

	raw	$m = 17$	$m = 25$	$m = 33$	$m = 41$
Remainder std (m/s^2)	–	0.578	1.012	1.372	1.572
Oracle recovery within one sample	0.687	0.980	0.989	0.993	0.998
Median period error (samples)	0.525	0.157	0.122	0.142	0.216

In numbers, oracle period recovery rises from 0.687 (raw) to 0.998 ($m = 41$). So, we selected $\mathbf{m = 41}$. Note that the choice depends on what we measure: $m = 41$ gives the best recovery within one sample, while $m = 25$ gives the smallest median period error.

Why a moving average suits this world. The drift changes slowly, over about 240 samples, while the vibration repeats every 14–36 samples. A 41-sample window is longer than every vibration cycle, so it averages each cycle out to about zero. Yet it is much shorter than 240, so the drift passes into the trend almost unchanged. The oracle check is a fair way to pick m because it tests exactly what we want: does the remainder show the true repeating pattern? It uses training data only, so validation and test data stay untouched.

Table 4: Output 2.3 – decomposition ablation, validation RMSE (m/s^2).

Model	Established	Held-out	Est. A / B / C	Held-out A / B / C
Raw Attention	0.436	0.659	0.412 / 0.462 / 0.433	0.677 / 0.619 / 0.678
Attention + decomposition	0.324	0.486	0.302 / 0.375 / 0.289	0.428 / 0.556 / 0.464

Effect on the model. Adding decomposition to Attention lowers validation RMSE by 26% on established stations ($0.436 \rightarrow 0.324$) and by 26% on Station 4 ($0.659 \rightarrow 0.486$). It improves every product.

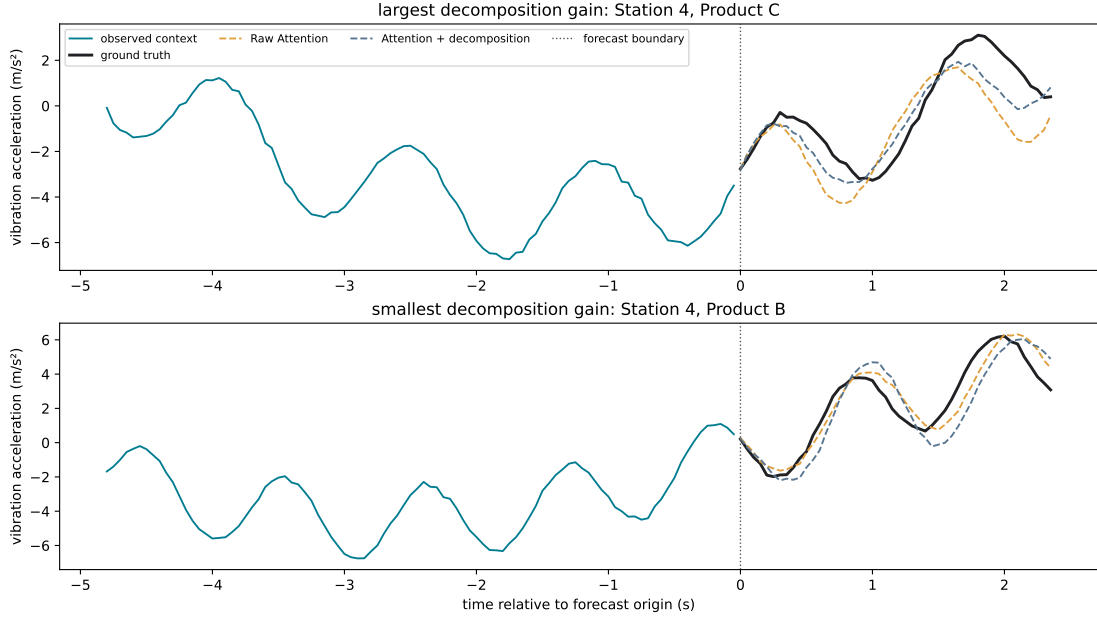


Figure 5: Output 2.3 – largest and smallest decomposition gains (Station 4).

Where it fails, and an alternative. A fixed moving average struggles with a sudden jump in level, e.g. a product changeover or an impact inside the window. It spreads the jump over 41 samples and leaks a false bump into the remainder. It would also fail if a vibration cycle were longer than the window. An alternative is STL/LOESS decomposition, which fits small local curves instead of a flat local average. It handles a curved trend better, but needs more computation and tuning.

Response 3 – Mixing by temporal delay

`aggregate_delays` gives the same answer as the worked example (35, 15, 25, 25). So $z_0 = 35$, and the shift runs in the right direction. It also passes the gradient check. Output 3.1 confirms that `delay_scores` matches a direct circular correlation (4, -4, 4, -4).

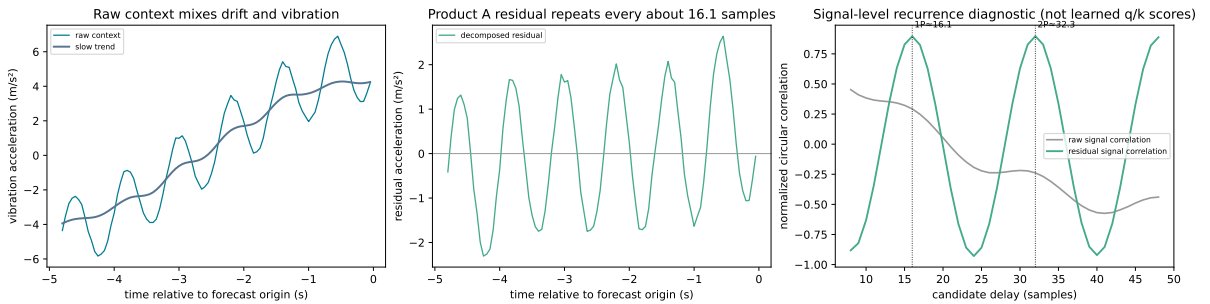


Figure 6: Output 3.2 – signal-level recurrence diagnostic (Station 2, Product A, $P \approx 16.1$). This is a correlation computed directly from the signal, not the model’s learned q/k scores.

Strongest lags. After removing the trend, the correlation peaks at lag 16 (0.896) and lag 32 (0.897). These match the wave’s period ($P \approx 16.1$) and twice the period (≈ 32.3). The dips at about 8, 24 and 40 are half a period apart, where the peaks of one copy line up with the troughs of the other.

Why decomposition helps. In the raw signal, the correlation just keeps dropping, from about 0.45 at lag 8 to -0.55 at lag 40, with no peak at the period. The slow ramp looks similar to itself at every small shift, so it hides the repeating wave. Once the trend is removed, an almost pure wave is left, and peaks appear only at multiples of the period. This is exactly what the delay mixer relies on.

If the useful information sits at a few repeating gaps, it makes sense to score L delays instead of all L^2 position pairs, and to combine shifted copies of the signal. This works best on the decomposed signal. In line with this, the delay mixer alone already halves Raw Attention’s error in Output 4.1 ($0.436 \rightarrow 0.214$).

Where delay mixing goes wrong. A one-off event, like a single knock from a loose part, never repeats. The delay mixer can only reuse shifted copies of the window, so it would either average the event away or copy it forward as false “echoes”. Similarly, if the speed changes quickly within one window (e.g. while the machine is speeding up), no small fixed set of delays fits the whole window.

Response 4 – Comparing designs and deploying

(a) Prediction (written before Output 4.1). I expect decomposition to help most, since it splits off the slow trend with a moving average and sends it to a simple linear head, leaving the encoder only the faster vibration.

Table 5: Validation error by model. RMSE in m/s^2 , MSE in $(\text{m/s}^2)^2$.

Model	Established		Held-out		Params	Fit (s)
	RMSE	MSE	RMSE	MSE		
Shared ridge	0.767	0.588	0.876	0.767	4,608	0.001
Sensor-specific ridge	0.768	0.590	n/a	n/a	13,824	0.001
Period-routed ridge	0.166	0.028	0.173	0.030	59,904	0.002
Raw Attention	0.436	0.190	0.659	0.434	368,368	206
Attention + decomposition	0.324	0.105	0.486	0.236	373,024	292
Raw delay mixer	0.214	0.046	0.338	0.114	368,370	776
Autoformer-inspired	0.197	0.039	0.244	0.059	373,026	852

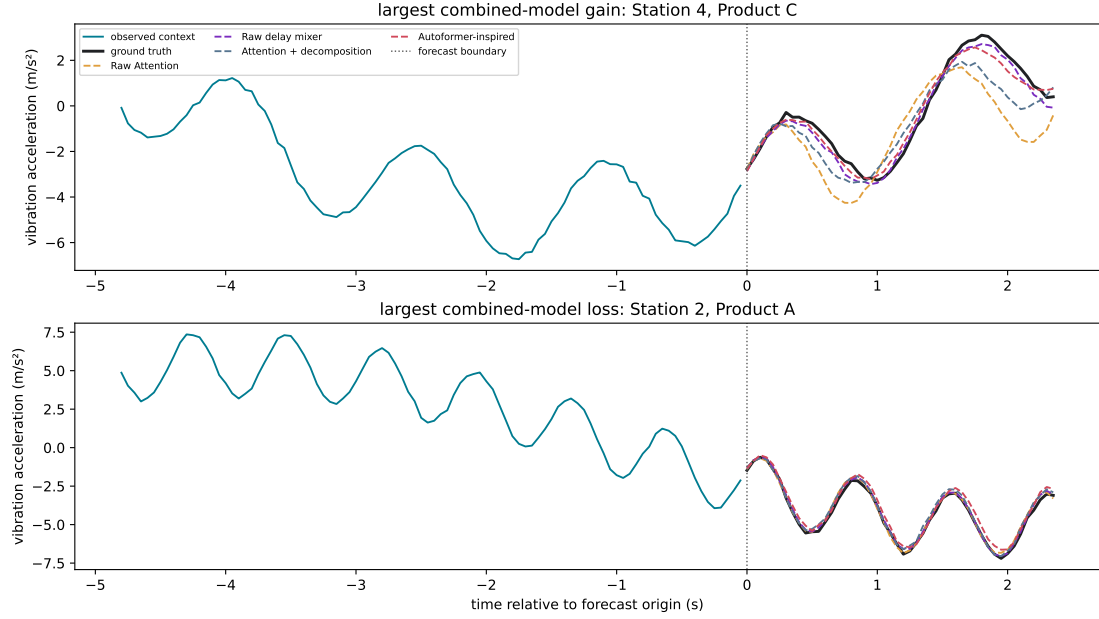


Figure 7: Output 4.1 – contrasting forecast cases for the four neural configurations.

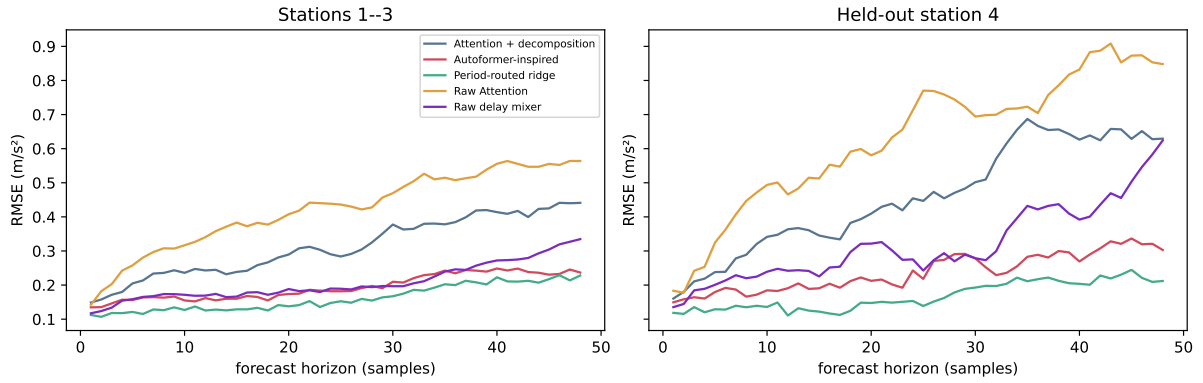


Figure 8: Output 4.2 – validation RMSE across the 48-step horizon.

(b) Validation results (Outputs 4.1–4.2).

Interpretation (≤ 200 words). On established stations, decomposition alone cuts Raw Attention’s error by about a quarter ($0.436 \rightarrow 0.324$), delay mixing alone by about half ($0.436 \rightarrow 0.214$), and using both gives the best result (0.197). However, the combined gain is smaller than the two gains added together. This suggests the two upgrades fix overlapping parts of the same drift-plus-repetition problem. On held-out Station 4, decomposition still helps a lot on top of delay mixing ($0.338 \rightarrow 0.244$). Contrary to my prediction, delay mixing was the stronger single upgrade.

Across the horizon, all models start with similar errors, and the errors grow the further ahead they forecast. Raw Attention gets worse the fastest, while Autoformer-inspired stays the most stable among the neural models. Period-routed ridge has the lowest error at every step. So, the simple linear model still beats every neural model.

(c) Deployment choices (fixed before Output 4.3).

- **Established stations:** For Stations 1–3, we chose Period-routed ridge because it gave the lowest validation error of all models (0.166, compared with 0.197 for the best neural model). It also uses about $6\times$ fewer parameters and fits almost instantly, while the neural model takes about 14 minutes.
- **Held-out Station 4:** For Station 4, we chose the same model because it again gave the lowest validation error (0.173, compared with 0.244) and the same cost advantage.

Table 6: Output 4.3 – selected models, validation and test.

Population	Model	Val. RMSE	Val. MSE	Test RMSE	Test MSE
Established	Period-routed ridge	0.166	0.028	0.161	0.026
Station 4	Period-routed ridge	0.173	0.030	0.183	0.033

Task 2 – Leaderboard Challenge (Autoformer)

We created three versions of the model, one for each leaderboard submission, and improved each one based on our validation results. The final model is a small Autoformer with 23,362 parameters. On top of the standard Autoformer, it has two additions:

- a **non-linear covariate head**, which lets the external data affect each forecast step directly;
- a **floor layer**, which stops the forecast from going below a realistic minimum value (14.7).

Data exploration and periodic structure

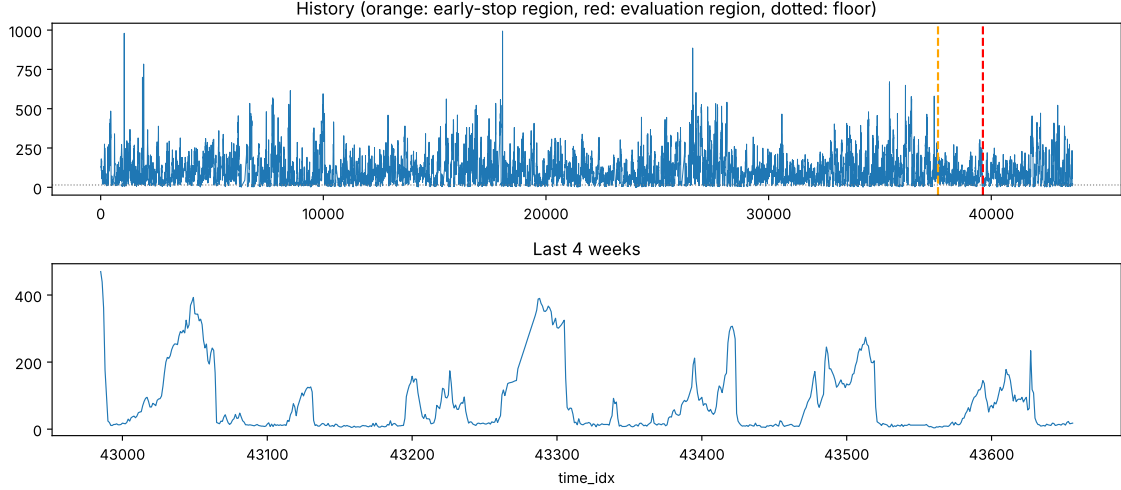


Figure 9: Task 2 history with the early-stop (orange) and evaluation (red) regions; the dotted line is the forecast floor (14.7).

Heavy tails. The values range from 0 to 994 and can jump by ten times within a few days. Exact zeros almost never occur (only 2 of 43,656 hours).

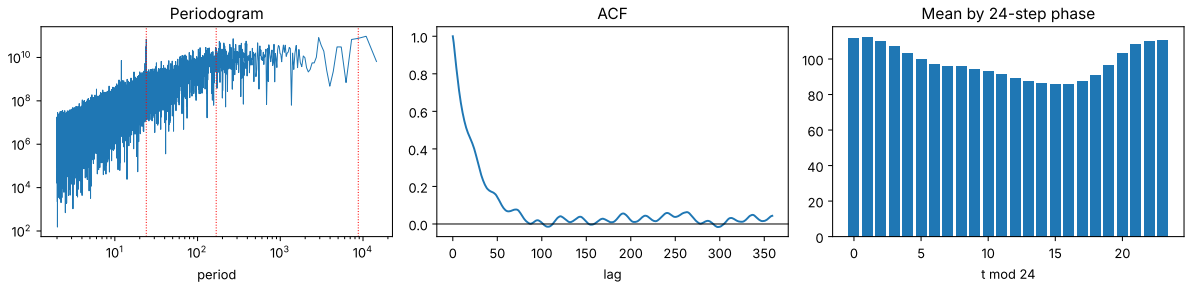


Figure 10: Periodogram, autocorrelation and mean by 24-step phase.

Repeating cycles. The periodogram peaks at 24 steps and near 8,730 (close to 8,766, the number of hours in a year). So, the data is most likely hourly, with a daily and a yearly cycle. The daily cycle is weak: the average value across the day only moves between 86 and 113, around an overall mean of 98. Since no dates are given, we turned these two cycles into sin/cos “phase” features, which replace the time stamps Autoformer normally uses.

Short memory. The autocorrelation is 0.97 one hour apart but drops to 0.40 after a day, 0.16 after two days and 0.03 after a week. So, the recent past says very little about most of the

168-hour forecast.

External file. It has six continuous features (A–F) and four on/off flags (G–J), where only one flag is on at a time. On their own, they are only weakly related to the target (strongest: D -0.25 , H -0.21 , A $+0.17$, J $+0.15$). So, whether they actually help had to be tested by experiment. To prepare them, we applied `log1p` to the very skewed features D–F, standardised the continuous features using the training data, and kept the flags as 0/1.

Validation protocol

Split. The data was split in time order. Training used steps 1–37,608, the next 12 weeks were used only for early stopping, and the final 24 weeks were kept for evaluation.

Scoring. Each of the 24 evaluation weeks was scored separately using MAE, RMSE and sMAPE, and the scores were then averaged. Weeks 12–23 are the most recent and most volatile, so they were treated as the closest match to the leaderboard week.

Seeds. Every configuration was run with 3 random seeds and reported as mean $\pm$ std. Configurations were also compared week by week to confirm that any difference was genuine.

Table 7: Naive baselines on the 24 evaluation weeks.

Baseline	MAE	RMSE	sMAPE (%)
Persistence (last value)	96.7	117.4	89.1
Seasonal naive (repeat last 24)	85.5	108.2	95.3
Mean of last 168	68.5	83.2	80.0
Training mean (flat line)	69.0	82.9	81.0

A few simple baselines were tested first. Predicting a flat line at the training mean outperformed repeating the last value by 34 RMSE. So, an RMSE of about 83 was set as the minimum any model had to beat.

Model design

- **Autoformer body.** The Auto-Correlation and series-decomposition layers are adapted from the official implementation (github.com/thuml/Autoformer, MIT licence; Wu et al., 2021) and wired as in its reference model: input decomposition (moving average, kernel 25); a decoder trend initialised from the window mean and a seasonal part from zeros; Auto-Correlation in the encoder and in both decoder attentions, keeping $k = \lfloor \ln L \rfloor = 5$ delays; decomposition after every attention and feed-forward block, with the trend accumulated through the decoder. The adapted files are in `Task 2 - Leaderboard/autoformer/`, with the source credited in each file header.
- **Size:** $d_{\text{model}} = 32$, 4 heads, 1 encoder and 1 decoder layer, $d_{ff} = 64$. Input length 168, label length 48, `pred_len` 168.
- **Embedding (my own):** a circular-conv value embedding plus a linear map of the marks (phase features + covariates), with no positional encoding. The encoder and decoder take separate marks, which is what the external-data ablation varies.
- **Exogenous head:** a term added at each forecast step, computed from the decoder marks over the horizon. Auto-Correlation aggregates time-shifted copies of the whole sequence, which blurs an effect tied to one step, and the head keeps that effect local. It is a 5-step convolution, a GELU and a 1×1 convolution (1,153 parameters), so it reacts to *changes* in the covariates, not only their levels.

- **Floor layer:** the last operation is $\max(\hat{y}, f)$ with $f = 14.7$, the 10th percentile of the training target.
- **Training:** MSE on the standardised target; AdamW (weight decay 10^{-4} , learning rate 3×10^{-4} , 3×10^{-3} for the head); cosine schedule; dropout 0.25; batch 64; training windows every 2 steps; early stopping with patience 4 (maximum 15 epochs).

Per model: **23,362** trainable parameters (22,209 body and embeddings + 1,153 head).

External-data ablation (final model, 3 seeds)

Table 8: With vs. without the optional file: mean $\pm$ std over 3 seeds, 24 evaluation weeks.

Configuration	MAE	RMSE	sMAPE (%)
Target only	68.2 ± 2.2	82.5 ± 1.7	78.9 ± 0.9
+ external (past only, encoder)	68.9 ± 2.4	82.8 ± 1.4	79.4 ± 1.1
+ external (past + horizon, encoder and decoder)	42.0 ± 2.4	56.8 ± 3.0	52.0 ± 0.3

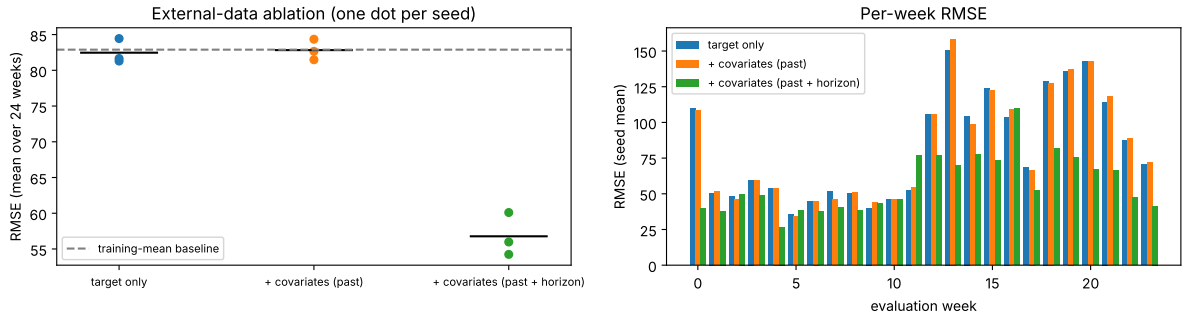


Figure 11: External-data ablation: RMSE per seed (left) and per evaluation week (right).

- **Covariates help only when they cover the horizon.** Past + horizon vs. target only: $\Delta\text{RMSE} = -25.7 \pm 11.3$, better on 18/24 weeks. Past only vs. target only: $+0.4 \pm 1.2$, better on 10/24, i.e. no effect.
- **Interpretation.** The covariates are measured on the same clock as the target and are known over the forecast window. Knowing them up to the forecast origin adds nothing, because the target’s own memory is short. Their value lies in describing conditions *during* the horizon, so they must enter the decoder.
- Without the horizon covariates, the Autoformer is no better than a flat line (82.5 vs. 82.9).

Submissions and design changes

Each change below was decided on the evaluation weeks before submitting; the leaderboard was used only to confirm.

Table 9: The three leaderboard submissions. Validation = RMSE over the 24 evaluation weeks (all / weeks 12–23) of exactly the configuration submitted.

#	Model	Val. all	Val. 12–23	P	E	Leaderboard RMSE	Rank
1	Linear head, 1 seed, no refit, clip at 0	63.8	79.3	22,224	4	93.82	~23
2	Non-linear head, 3 seeds + refit, floor after prediction	56.0	71.4	70,086	18	71.48	8
3	Non-linear head, 3 seeds + refit, floor layer	54.3	68.3	70,086	18	67.22	3

Table 10: Design checks on the past + horizon configuration (3 seeds, RMSE mean $\pm$ std over seeds).

Configuration	All 24 weeks	Weeks 12–23
Linear head, dropout 0.1, LR halved each epoch (v1)	60.50 $\pm$ 2.48	75.87 $\pm$ 2.43
Non-linear head, floored after prediction (v2)	57.85 $\pm$ 2.42	72.58 $\pm$ 2.43
Non-linear head, floor layer (v3)	56.77 $\pm$ 2.46	69.89 $\pm$ 2.37

- **First version.** It used a linear covariate head, a single seed and no refit, with negative values clipped at 0. It scored 93.8 on the leaderboard, against a validation mean of 60.5. The calm early weeks had made the average look better than it was. On the recent weeks alone, the same model scored 79.3, with weekly RMSE ranging from 33 to 101. So, from then on, models were selected on weeks 12–23 only.
- **Second version.** Three changes were made here:
 - Regularisation was tried first but did not help ($+0.67 \pm 1.36$, better on only 6 of 24 weeks). Every run still peaked at epoch 1 or 2, so the early stopping was not caused by overfitting.
 - The linear head was replaced with a non-linear one. This lowered RMSE by 3.05 ± 4.41 and was better on 17 of 24 weeks, since the largest errors happen at sudden changes, which a 5-step convolution can capture.
 - The models were retrained on the full history, which improved every seed (-0.53 RMSE), and three seeds were averaged, which lowered RMSE by a further 1.5–2.5. Clipping at 0 was also replaced with a floor of 14.7, applied after prediction.

It scored 71.48 on the leaderboard (8th).

- **Third version.** The only change was moving the floor inside the model as a layer, so the model learns with it during training. Compared with the second version, this lowered RMSE by 1.08 ± 2.63 , and by more on the seed-averaged recent weeks ($71.40 \rightarrow 68.34$, with MAE falling from 45.7 to 42.0). It scored 67.22 on the leaderboard (3rd).

The forecast floor

A forecast floor is simply the lowest value the model is allowed to predict. The target here is never negative and almost never zero, so a forecast below a realistic minimum can only be wrong. In the first version, negative outputs were clipped at 0, which gave long runs of exact zeros at the end of the forecast. On validation weeks where this happened, the true values averaged around 14, not 0. So, in the second version, the floor was raised to 14.7, the 10th percentile of the training data, and applied after prediction.

In the third version, the same floor was moved inside the model as a layer, so the model trained with it rather than having its output trimmed afterwards. This improved the score, but the forecast now sits exactly at 14.70 for 44 of 168 hours, at the start and end. In these hours the external data points to a low period, and the model pushes its output well below the floor.

A natural next step would be to let the model learn the floor, either by learning the floor level itself during training or by using a smooth floor that still passes a learning signal below 14.7. This would let it predict values such as 18 or 25 in low periods instead of a flat 14.70. However, since the third version already scored well and matched its validation estimate, and the expected gain was small, this was left untested.

Final model: validation and forecast

Table 11: Final model on the 24 evaluation weeks.

Seed	MAE	RMSE	sMAPE (%)	RMSE weeks 12–23	Best epoch	Epochs run
0	44.52	60.10	51.74	72.89	1	5
1	41.63	55.98	51.93	69.72	1	5
2	39.80	54.24	52.28	67.08	1	5
3-seed average	40.01	54.26	49.32	68.34	—	—

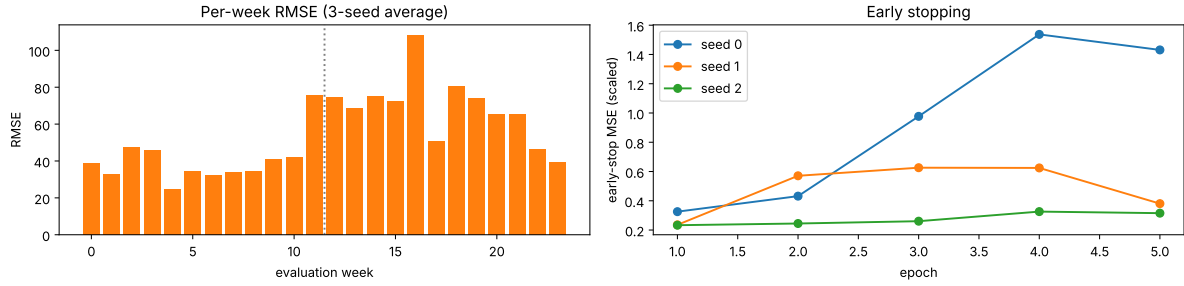


Figure 12: Final model: per-week RMSE of the 3-seed average (left); early-stopping loss per seed (right).

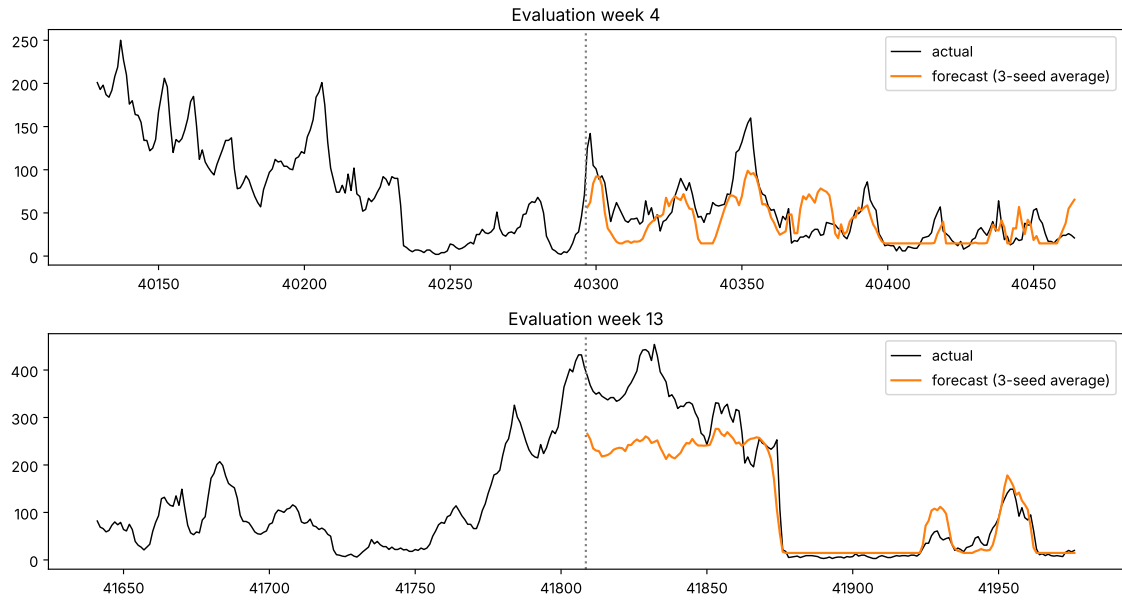


Figure 13: Two evaluation weeks: actual vs. the 3-seed average forecast.

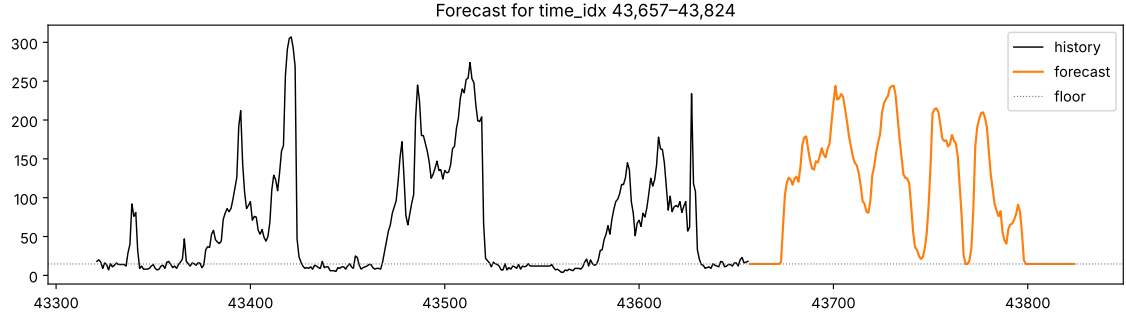


Figure 14: Submitted forecast for `time_idx` 43,657–43,824.

Final submission (third version). Each of the three models was retrained on the full history (all 43,656 steps) for 1 epoch, the best epoch found by early stopping. Their three forecasts were then averaged. This gives $P = 3 \times 23,362 = 70,086$ parameters. Each model ran 5 epochs during early stopping plus 1 refit epoch, so $E = 3 \times (5 + 1) = 18$.

Limitations. It was found that every run reached its best result at epoch 1, which suggests that the main Autoformer part adds less than its size implies. Instead, much of the gain comes from the covariate head, which agrees with Zeng et al. (2023).

References

- H. Wu, J. Xu, J. Wang, M. Long. Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. *NeurIPS*, 2021. Code: <https://github.com/thuml/Autoformer> (MIT licence).
- A. Zeng, M. Chen, L. Zhang, Q. Xu. Are Transformers Effective for Time Series Forecasting? *AAAI*, 2023.

Appendix – Use of Generative AI

Claude (Anthropic) was used as an assistant throughout the assignment:

- **Understanding the tasks:** explaining the assignment requirements and concepts in Task 1 and Task 2.
- **Code:** implementing and fixing functions in Task 1 (`RawAttentionForecaster.forward`, `SeriesDecomposition.forward`, `aggregate_delays`); building the Task 2 training and validation pipeline around the cited thuml Autoformer layers; organising the repository.
- **Graphs and tables:** producing the Task 2 figures and filling result tables from my runs.
- **Architectural variations:** suggesting and testing changes on validation (regularisation, a non-linear covariate head, full-history refit, seed averaging, the floor layer).
- **Writing:** drafting response text and the Task 2 section from my results.

All decisions were my own: the Task 1 predictions and deployment choices, which approach to adapt in Task 2, which changes to try, what to submit to the leaderboard, and the final content of this report.